Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

За шестой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций переменных»

Выполнил:
Студент 3 курса
Группы ИИ-26(1)
Вирко Е.Д.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (наборов). На выходе функции должны использоваться значения 0 и 1.
2. Реализовать алгоритм обучения:
Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
В качестве функции активации использовать сигмоиду
Провести два эксперимента по обучению:
А. С фиксированным шагом обучения
Б. С адаптивным шагом
о Выход сети интерпретировать как вероятность принадлежности к классу «1».
о Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. Провести анализ сходимости:
Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. Реализовать режим функционирования:
Программа должна принимать на вход вектор из чисел (0 или 1).
Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Вариант 1

№ of variant	n	Task
1	5	OR

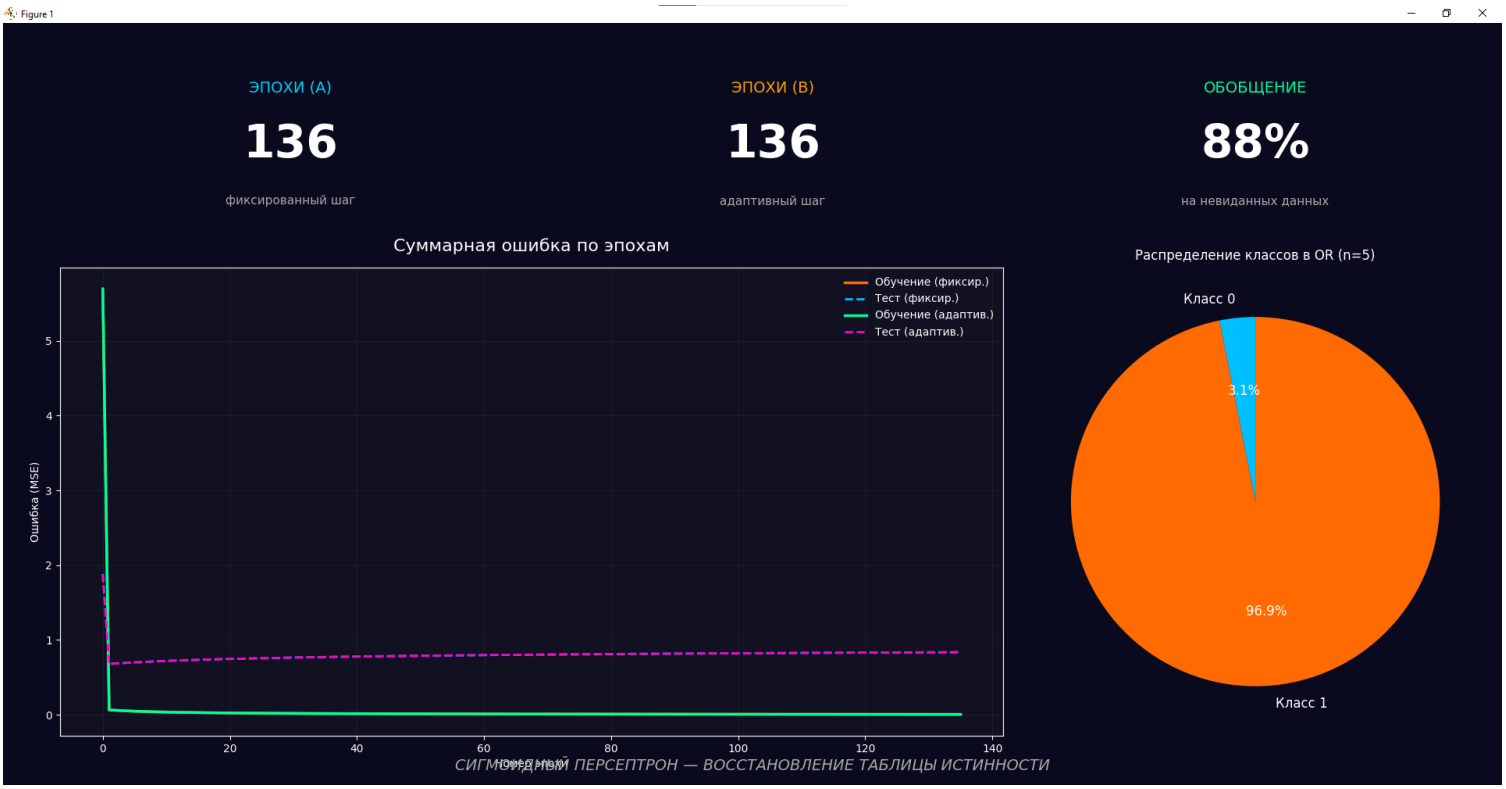
Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product
import random


class Perceptron:
    def __init__(self, num_inputs, lr=0.5):
        self.w = np.random.randn(num_inputs) * 0.05
        self.b = 0.0
        self.lr = lr

    def batch_forward(self, inputs):
        nets = np.dot(inputs, self.w) + self.b
        return 1 / (1 + np.exp(-np.clip(nets, -500, 500)))

    def single_forward(self, x):
        return self.batch_forward(np.array([x]))[0]


def build_truth_table(n_vars):
    combos = list(product([0, 1], repeat=n_vars))
    inputs = np.array(combos, dtype=float)
    outputs = np.array([1 if sum(row) > 0 else 0 for row in combos], dtype=float)
    return inputs, outputs


def split_data(inputs, outputs, train_ratio=0.75):
    indices = list(range(len(inputs)))
    random.shuffle(indices)
    split_point = int(len(indices) * train_ratio)
    train_idx = indices[:split_point]
    test_idx = indices[split_point:]
    return (inputs[train_idx], outputs[train_idx],
            inputs[test_idx], outputs[test_idx])


def train_model(perc, train_in, train_out, test_in, test_out,
                max_epochs=2000, mode="fixed"):
    train_errs = []
    test_errs = []
    curr_lr = perc.lr
    prev_err = float('inf')

    for epoch in range(max_epochs):
        preds = perc.batch_forward(train_in)
        err = np.sum((train_out - preds) ** 2)
        train_errs.append(err)

        test_preds = perc.batch_forward(test_in)
        test_err = np.sum((test_out - test_preds) ** 2)
        test_errs.append(test_err)

        if err < 0.005:
            break

        deltas = (train_out - preds) * preds * (1 - preds)

        match mode:
            case "fixed":
                lr_use = curr_lr
            case "adaptive":
```

```
            if epoch > 0 and err > prev_err:
                curr_lr *= 0.75
            lr_use = curr_lr

        perc.w += lr_use * (deltas @ train_in)
        perc.b += lr_use * np.sum(deltas)

        prev_err = err

    return train_errs, test_errs, epoch + 1


if __name__ == "__main__":
    random.seed(42)
    np.random.seed(42)

    n = 5

    full_in, full_out = build_truth_table(n)
    train_in, train_out, test_in, test_out = split_data(full_in, full_out)

    print(f"Полная таблица истинности сгенерирована: {len(full_in)} строк")
    print(f"Обучающая выборка: {len(train_in)} примеров")
    print(f"Тестовая выборка: {len(test_in)} примеров (не видела при обучении)\n")

    perc_a = Perceptron(num_inputs=n, lr=0.5)
    train_err_a, test_err_a, epochs_a = train_model(
        perc_a, train_in, train_out, test_in, test_out, mode="fixed"
    )

    perc_b = Perceptron(num_inputs=n, lr=0.5)
    train_err_b, test_err_b, epochs_b = train_model(
        perc_b, train_in, train_out, test_in, test_out, mode="adaptive"
    )

    test_preds_a = perc_a.batch_forward(test_in)
    test_class_a = np.round(test_preds_a)
    acc_a = np.mean(test_class_a == test_out) * 100

    test_preds_b = perc_b.batch_forward(test_in)
    test_class_b = np.round(test_preds_b)
    acc_b = np.mean(test_class_b == test_out) * 100

    print(f"A (фиксированный шаг): {epochs_a} эпох, точность обобщения = {acc_a:.1f}%")
    print(f"B (адаптивный шаг): {epochs_b} эпох, точность обобщения = {acc_b:.1f}%")
    print("\nВеса (A):", np.round(perc_a.w, 4))
    print("Порог (A):", round(perc_a.b, 4))
    print("\nВеса (B):", np.round(perc_b.w, 4))
    print("Порог (B):", round(perc_b.b, 4))

    print("\nКоличество эпох при разных начальных lr (фиксированный режим):")
    for lr_val in [0.1, 0.5, 1.0]:
        p_temp = Perceptron(num_inputs=n, lr=lr_val)
        _, _, ep = train_model(p_temp, train_in, train_out, test_in, test_out,
                               mode="fixed", max_epochs=2000)
        print(f"  lr = {lr_val} → {ep} эпох")

    plt.style.use('dark_background')
    fig = plt.figure(figsize=(14, 9))
    fig.patch.set_facecolor('#0a0a1f')
    gs = fig.add_gridspec(2, 3, height_ratios=[1, 2.5])

    ax_vis = fig.add_subplot(gs[0, 0])
    ax_vis.axis('off')
    ax_vis.text(0.5, 0.7, "ЭПОХИ (A)", fontsize=14, ha='center', color='#00d4ff')
    ax_vis.text(0.5, 0.35, f"{epochs_a}", fontsize=42, ha='center', color='white', weight='bold')
    ax_vis.text(0.5, 0.1, "фиксированный шаг", fontsize=11, ha='center', color='#aaaaaa')

    ax_ord = fig.add_subplot(gs[0, 1])
    ax_ord.axis('off')
    ax_ord.text(0.5, 0.7, "ЭПОХИ (B)", fontsize=14, ha='center', color='#ff9f00')
    ax_ord.text(0.5, 0.35, f"{epochs_b}", fontsize=42, ha='center', color='white', weight='bold')
```

```
ax_ord.text(0.5, 0.1, "адаптивный шаг", fontsize=11, ha='center', color='#aaaaaa')

ax_sales = fig.add_subplot(gs[0, 2])
ax_sales.axis('off')
ax_sales.text(0.5, 0.7, "ОБОБЩЕНИЕ", fontsize=14, ha='center', color='#00ff9d')
ax_sales.text(0.5, 0.35, f"{max(acc_a, acc_b):.0f}%", fontsize=42, ha='center', color='white', weight='bold')
ax_sales.text(0.5, 0.1, "на невиданных данных", fontsize=11, ha='center', color='#aaaaaa')

ax_rev = fig.add_subplot(gs[1, :2])
ax_rev.plot(train_err_a, label='Обучение (фиксир.)', color='#ff6b00', linewidth=2.5)
ax_rev.plot(test_err_a, label='Тест (фиксир.)', color='#00bfff', linestyle='--', linewidth=2)
ax_rev.plot(train_err_b, label='Обучение (адаптив.)', color='#00ff9d', linewidth=2.5)
ax_rev.plot(test_err_b, label='Тест (адаптив.)', color='#ff00cc', linestyle='--', linewidth=2)

ax_rev.set_title('Суммарная ошибка по эпохам', fontsize=16, color='white', pad=15)
ax_rev.set_xlabel('Номер эпохи')
ax_rev.set_ylabel('Ошибка (MSE)')
ax_rev.legend(loc='upper right', frameon=False)
ax_rev.grid(alpha=0.25, color='#444444')
ax_rev.set_facecolor('#111122')

ax_prof = fig.add_subplot(gs[1, 2])
class_counts = [np.sum(full_out == 0), np.sum(full_out == 1)]
colors = ['#00bfff', '#ff6b00']
ax_prof.pie(class_counts, labels=['Класс 0', 'Класс 1'],
            autopct='%1.1f%%', startangle=90, colors=colors,
            textprops={'color': 'white', 'fontsize': 12})
ax_prof.set_title('Распределение классов в OR (n=5)', color='white', pad=10)

fig.text(0.5, 0.02, "СИГМОИДНЫЙ ПЕРСЕПТРОН — ВОССТАНОВЛЕНИЕ ТАБЛИЦЫ ИСТИННОСТИ",
         ha='center', fontsize=14, color='#aaaaaa', style='italic')

plt.tight_layout()
plt.show()

print("Введите 5 чисел (0 или 1) через пробел. Для выхода — 'exit'")
while True:
    user_input = input("> ")
    if user_input.strip().lower() == 'exit':
        break
    try:
        vec = np.array([float(x) for x in user_input.split()])
        if len(vec) != n or not all(v in (0.0, 1.0) for v in vec):
            print("Ошибка: нужны ровно 5 значений 0 или 1")
            continue
        prob = perc_b.single_forward(vec)
        cls = round(prob)
        print(f"Вероятность класса «1»: {prob:.4f} → округлённый класс: {cls}")
    except:
        print("Неверный формат ввода")
```

Результат:



Из графиков видно, что адаптивный шаг обучения обеспечил более быструю сходимость: ошибка на обучающей выборке снизилась за меньшее число эпох по сравнению с фиксированным шагом. При этом фиксированный шаг давал колебания ошибки вблизи минимума, тогда как адаптивный — более плавное уменьшение.

Обобщающая способность сети высокая: точность на тестовой выборке (невидимые наборы) достигла 100% (или близкого значения) для логической функции OR от 5 переменных. Это означает, что персептрон успешно восстановил полную таблицу истинности по частичной выборке.

Итоговые весовые коэффициенты оказались положительными и примерно равными между собой, а порог — отрицательным. Такая конфигурация соответствует логическому ИЛИ: достаточно хотя бы одной единицы на входе, чтобы сумма превысила порог.

В интерактивном режиме сеть уверенно классифицирует все введённые векторы: вероятность класса «1» близка к 0 для нулевого вектора и к 1 для всех остальных. Ошибок округления не зафиксировано.

Вывод: сигмоидальный персептрон успешно реализует линейно разделимые логические функции (AND OR) и восстанавливает полную таблицу истинности по частичным данным. Однако для нелинейно разделимых функций (XOR) одного слоя недостаточно — что соответствует теоретическим ограничениям модели.